

From Proof Objects to Linear Proof Texts

AITP 2026, Aussois 30 August to 4 September 2026

Aarne Ranta

Department of Computer Science and Engineering
Chalmers University of Technology and University of Gothenburg

aarne.ranta@cse.gu.se

Jan von Plato

Department of Philosophy
University of Helsinki

jan.vonplato@helsinki.fi



NextReason
MALINCA
HERITAGE



Context: informalization

The ultimate syntactic sugar: ordinary mathematical language

```
Thm04 : (u : Elem Vector) -> (v : Elem Vector) ->  
  Proof (orthogonal u v) ->  
    Proof (Eq Real (length (vectorPlus u v))  
      (sqrt (plus (square (length u)) (square (length v)))))).
```

Thm04. For all vectors u and v , if $u \perp v$, then

$$\|u + v\| = \sqrt{\|u\|^2 + \|v\|^2}.$$

Variations in ordinary mathematical language

```
Thm04 : (u : Elem Vector) -> (v : Elem Vector) ->  
  Proof (orthogonal u v) ->  
    Proof (Eq Real (length (vectorPlus u v))  
      (sqrt (plus (square (length u)) (square (length v))))).
```

```
$ RunInformath -variations tmp/t4.dk | wc -l  
2542
```

Thm04. For all vectors u and v , if $u \perp v$, then

$$\|u + v\| = \sqrt{\|u\|^2 + \|v\|^2}.$$

Thm04. Let u and v be vectors. Assume that u is orthogonal to v . Then the length of the sum of u and v is equal to the square root of the sum of the square of the length of u and the square of the length of v .

Uses of informalization

Documenting formal mathematics

- Useful for documenting formal mathematics (Coscoy et al. 1994)

Formalizing informal mathematics

- via parsing with reversible grammars (CNL approach)
- via generating synthetic data for neural models (Wang et al. 2018)

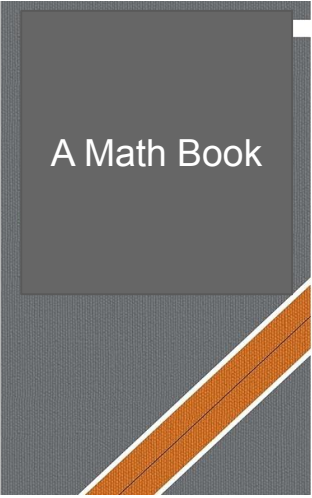
Uses of informalization

Documenting formal mathematics

- Useful for documenting formal mathematics (Coscoy et al. 1994)
- Indispensable for explaining AI generated code (Urban et al. 2026)

Formalizing informal mathematics

- via parsing with reversible grammars (CNL approach)
- via generating synthetic data for neural models (Wang et al. 2018)



A Math Book

LLM autoformalization

```
t0 a (t0 b c)) (t0 (t0 a b) c)) => match_Aop a1
a2 (plus Type2 (nd 1)) (___ : Aop a1 a2 => univ
Type2) (u0 : (___ : a1 -> __1 : a1 -> a1) => u1
: (a : a1 -> Eq (u0 a2 a) a) => u2 : (a : a1 ->
Eq (u0 a a2) a) => u3 : (a : a1 -> b : a1 -> c
: a1 -> Eq (u0 a (u0 b c)) (u0 (u0 a b) c)) =>
prod (plus Type1 (nd 1)) Type1 (univ Type1) (P
: Type1 => prod Type1 Type1 (prod Prop Type1
(Eq (R0 (prod Set Set a1 (___ : a1 => prod Set
Set a1 (___1 : a1 => a1))) t0) u0) (e0 : Eq (R0
(prod Set Set a1 (___ : a1 => prod Set Set a1
(___1 : a1 => a1))) t0) u0 => prod Prop Type1
(Eq (R1 (prod Set Set a1 (___ : a1 => prod Set
Set a1 (___1 : a1 => a1))) t0 (x_19 : (___ : a1
-> __1 : a1 -> a1) => _x_20 : Eq t0 x_19 => (x0
: (___ : a1 -> __1 : a1 -> a1) => p0 : Eq t0 x0
=> prod Set Prop a1 (a : a1 => Eq (x0 a2 a) a))
x_19 _x_20) t1 u0 e0) u1) (e1 : Eq (R1 (prod
Set Set a1 (___ : a1 => prod Set Set a1 (___1 :
a1 => a1))) t0 (x_19 : (___ : a1 -> __1 : a1 ->
a1) => _x_20 : Eq t0 x_19 => (x0 : (___ : a1 ->
__1 : a1 -> a1) => p0 : Eq t0 x0 => prod Set
Prop a1 (a : a1 => Eq (x0 a2 a) a)) x_19 _x_20)
t1 u0 e0) u1 => prod Prop Type1 (Eq (R2 (prod
Set Set a1 (___ : a1 => prod Set Set a1 (___1 :
a1 => a1))) t0 (x0 : (___ : a1 -> __1 : a1 ->
a1) => ___ : Eq t0 x0 => (x01 : (___1 : a1 -> __2
: a1 -> a1) => p0 : Eq t0 x01 => prod Set Prop
a1 (a : a1 => Eq (x01 a2 a) a)) x0 ___) t1 (x0 :
(___ : a1 -> __1 : a1 -> a1) => p0 : Eq t0 x0 =>
x1 : (x01 : (___ : a1 -> __1 : a1 -> a1) => p01
: Eq t0 x01 => prod Set Prop a1 (a : a1 => Eq
(x01 a2 a) a)) x0 p0 => ___ : Eq (R1 (prod Set
Set a1 (___ : a1 => prod Set Set a1 (___1 : a1 =>
```

A Math Book

LLM autoformalization

```
t0 a (t0 b c)) (t0 (t0 a b) c)) => match_Aop a1
a2 (plus Type2 (nd 1)) (___ : Aop a1 a2 => univ
Type2) (u0 : (___ : a1 -> __1 : a1 -> a1) => u1
: (a : a1 -> Eq (u0 a2 a) a) => u2 : (a : a1 ->
Eq (u0 a a2) a) => u3 : (a : a1 -> b : a1 -> c
: a1 -> Eq (u0 a (u0 b c)) (u0 (u0 a b) c)) =>
prod (plus Type1 (nd 1)) Type1 (univ Type1) (P
: Type1 => prod Type1 Type1 (prod Prop Type1
(Eq (R0 (prod Set Set a1 (___ : a1 => prod Set
Set a1 (__1 : a1 => a1))) t0) u0) (e0 : Eq (R0
(prod Set Set a1 (___ : a1 => prod Set Set a1
(__1 : a1 => a1))) t0) u0 => prod Prop Type1
(Eq (R1 (prod Set Set a1 (___ : a1 => prod Set
Set a1 (__1 : a1 => a1))) t0 (x_19 : (___ : a1
-> __1 : a1 -> a1) => _x_20 : Eq t0 x_19 => (x0
: (___ : a1 -> __1 : a1 -> a1) => p0 : Eq t0 x0
=> prod Set Prop a1 (a : a1 => Eq (x0 a2 a) a))
x_19 _x_20) t1 u0 e0) u1) (e1 : Eq (R1 (prod
Set Set a1 (___ : a1 => prod Set Set a1 (__1 :
a1 => a1))) t0 (x_19 : (___ : a1 -> __1 : a1 ->
a1) => _x_20 : Eq t0 x_19 => (x0 : (___ : a1 ->
__1 : a1 -> a1) => p0 : Eq t0 x0 => prod Set
Prop a1 (a : a1 => Eq (x0 a2 a) a)) x_19 _x_20)
t1 u0 e0) u1 => prod Prop Type1 (Eq (R2 (prod
Set Set a1 (___ : a1 => prod Set Set a1 (__1 :
a1 => a1))) t0 (x0 : (___ : a1 -> __1 : a1 ->
a1) => ___ : Eq t0 x0 => (x01 : (__1 : a1 -> __2
: a1 -> a1) => p0 : Eq t0 x01 => prod Set Prop
a1 (a : a1 => Eq (x01 a2 a) a)) x0 ___) t1 (x0 :
(___ : a1 -> __1 : a1 -> a1) => p0 : Eq t0 x0 =>
x1 : (x01 : (___ : a1 -> __1 : a1 -> a1) => p01
: Eq t0 x01 => prod Set Prop a1 (a : a1 => Eq
(x01 a2 a) a)) x0 p0 => ___ : Eq (R1 (prod Set
Set a1 (___ : a1 => prod Set Set a1 (__1 : a1 =>
```

symbolic
informalization

Thm03. $\mathbb{Q}$ is denumerable.

Thm03a.

$$|N| = |\mathbb{Q}|.$$

Thm04. Let u and v be vectors. Then $u \perp v$ implies $\|u + v\| = \sqrt{\|u\|^2 + \|v\|^2}$.

Thm07. Let $p, q \in \mathbb{N}$. Then if p and q are prime, then

$$\binom{p}{q} \binom{q}{p} = (-1)^{\frac{p-1}{2} \frac{q-1}{2}}.$$

Thm09. Let c be a circle. Let r be a real number. Assume that $r = \text{radius}(c)$. Then the area of c is equal to $\pi \times r^2$.

Thm10FermatLittle. Let p be a natural number. Assume that p is prime. Let a be an integer. Then there exists an integer q , such that $a^p - a = p \times q$.

Thm11. $p \geq n$ and p is prime for a natural number p for all natural numbers n .

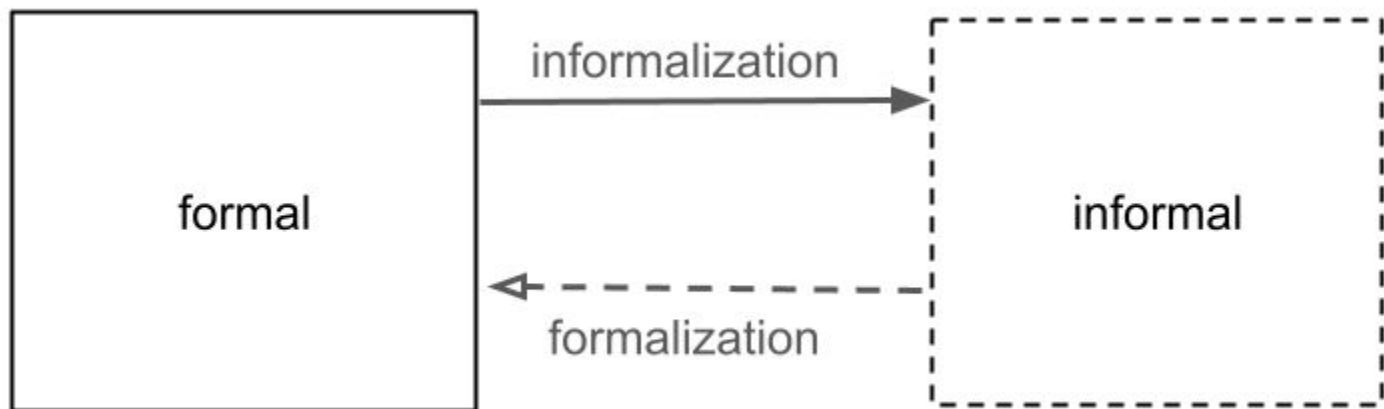
Thm19. $n = a^2 + b^2 + c^2 + d^2$ for some natural numbers a, b, c and d for all natural numbers n .

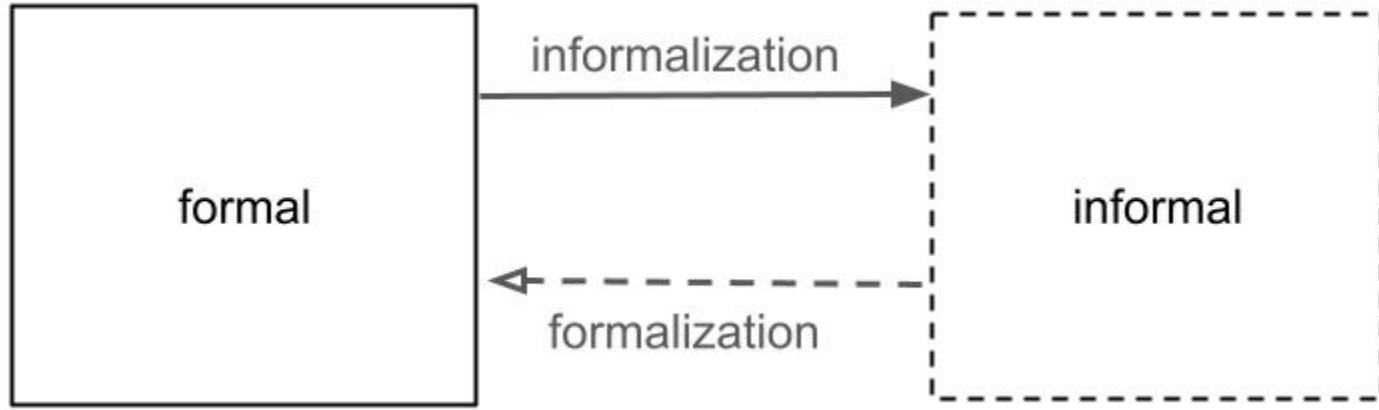
Thm20a. Let p be a natural number. Assume that p is prime. Let k be a natural number. Assume that $p = 4 \times k + 1$. Then there exists a natural number x , such that there exists a natural number y , such that $p = x^2 + y^2$.

Thm20b. Let p be a natural number. Assume that p is prime. Assume that $p \equiv 1 \pmod{4}$. Then there exists a natural number x , such that there exists a natural number y , such that $p = x^2 + y^2$.

Thm22. It is not the case that $\mathbb{R}$ is denumerable.

Mock-up example: the code and the text do not correspond to each other.



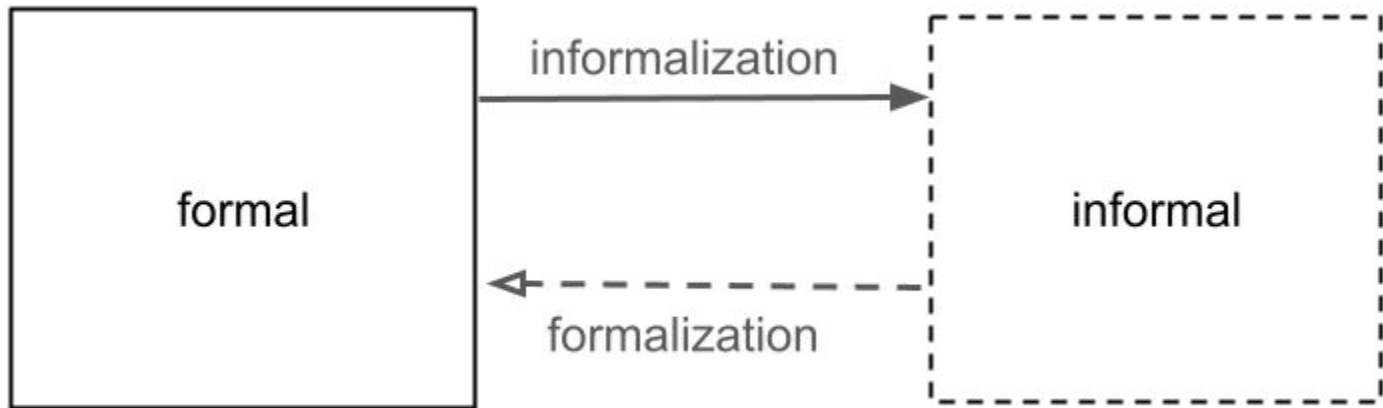


formal language: precisely defined

- possible to check
- possible to cover

informal language: not precisely defined

- not possible to check
- not possible to cover



formal language: precisely defined

- possible to check
- possible to cover

LLM formalization

- complete coverage
- results can be checked

rule-based informalization

- complete coverage
- correct by construction

informal language: not precisely defined

- not possible to check
- not possible to cover

The project Informath: goals

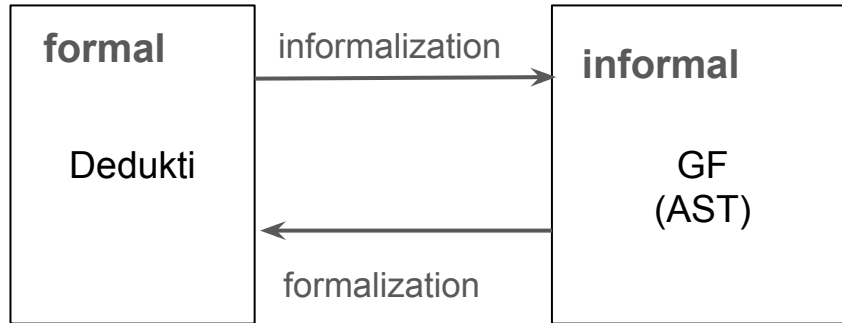
Semantic completeness: everything expressible in type theory can be translated to Informath natural language.

Semantic soundness: everything expressible in Informath can be translated to type theory.

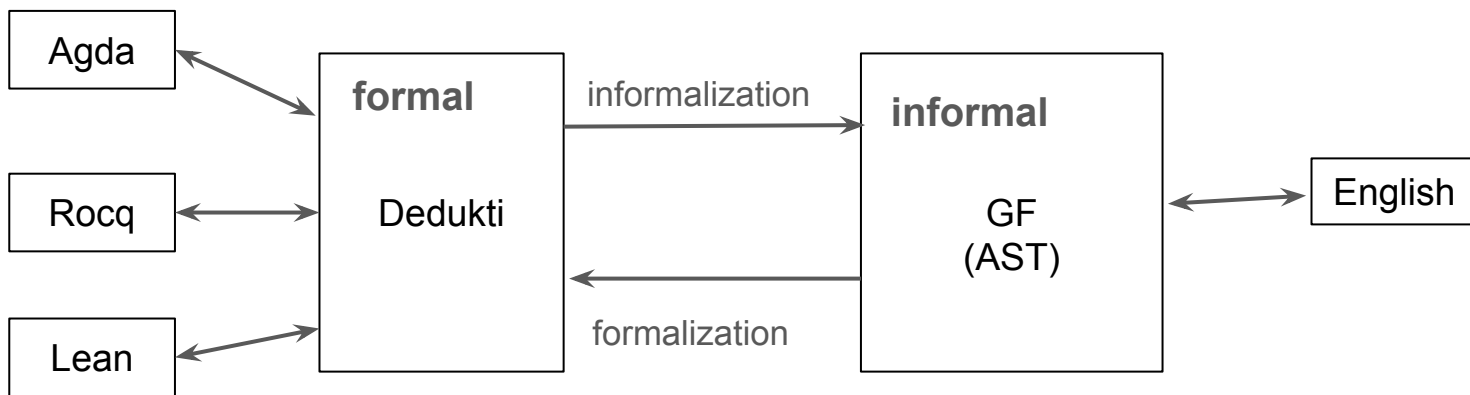
Linguistic completeness: everything attested in mathematical texts should be eventually covered by Informath.

Linguistic soundness: nothing should be added to Informath unless it is attested in mathematical texts.

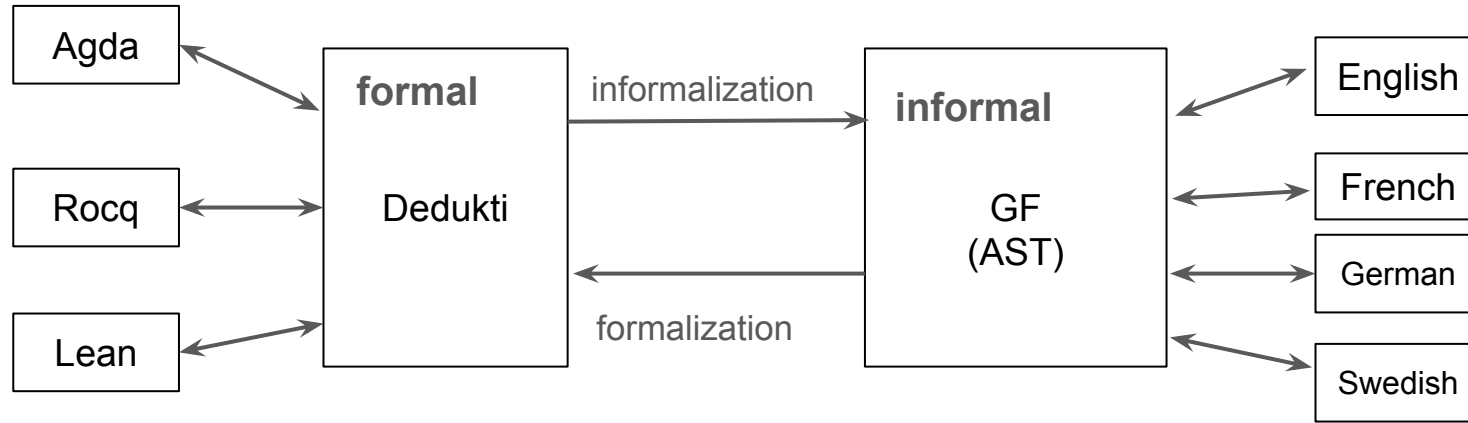
The Informath project



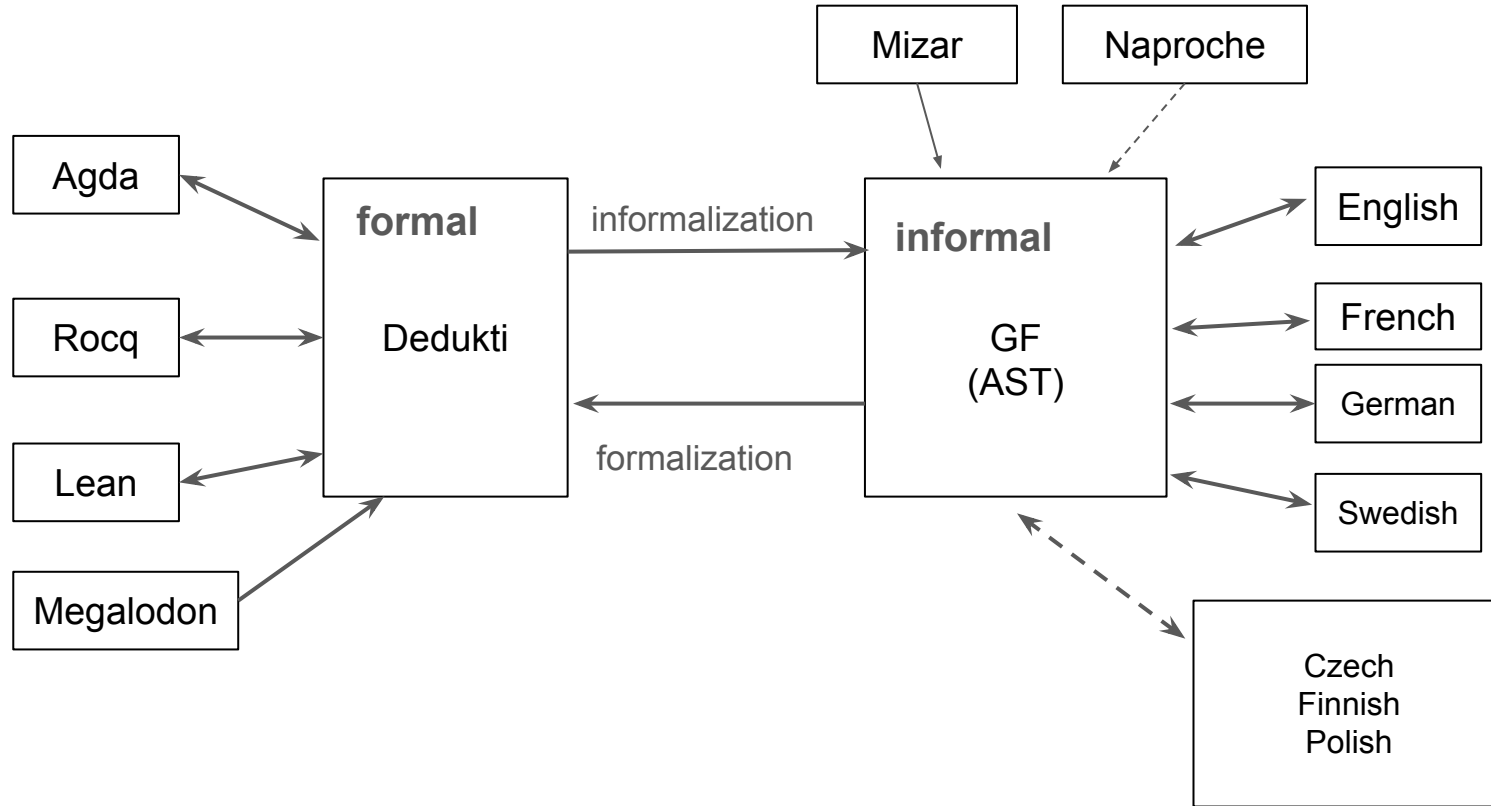
The Informath project



The Informath project



The Informath project



Interlingua: pros and cons

- + shared core algorithms:
 - type checking (Dedukti, GF)
 - language variations (GF)
 - formal-informal translations
- loss of structure
 - choice of proof steps and tactics
- heavy encoding of infrastructure in Dedukti
 - universes, coercions : functions may get 3 extra argument places

Adding proofs to Informath

Baseline: proof objects

Informath is already complete w.r.t. Dedukti, including proofs

- identifiers and their applications:

$$(f\ p_1 \dots p_n) : P \implies p_1 \dots p_n. \text{ By } f, P.$$

- abstractions:

$$B_1 \Rightarrow \dots \Rightarrow B_n \Rightarrow (p : P) \implies B_1 \dots B_n.p. \text{ We conclude } P.$$

But the proof objects are **HUGE**

→ texts become **GARGANTUAN**

Challenge: invert the de Bruijn factor

de Bruijn: formalization blow up 10-20 times

inversion: informalization should shrink it by 90 to 95%

What helps

The syntax and vocabulary in **proof steps** is already covered

- proofs rarely introduce new concepts: they use already defined ones

Informath grammar contains structures for proof steps

- assumptions
- conclusions
- declarations of goals

Prior art

Coscoy, Kahn and Théry 1995

- extract text from Coq proof objects
- works as an optimizing compiler
 - syntax-directed translation
 - meaning-preserving optimizations

What we add here

Don't informalize proof objects directly:

Convert them to sequences of proof steps first.

Gentzen to Jaśkowski

Table 2. Linearization of tree derivations

1. Number the open assumptions in a tree, from left to right in the obvious order, with each repetition of an assumption getting the same number, and let these numbers be 1 to n .
2. Continue the numbering of the formulas from formula 1 down until a two-premiss rule is reached, and repeat this in left to right order until formula n , with the last formula numbered being m .

Gentzen to Jaśkowski and Back: Algorithmic Translation...

3. If both branches of a two-premiss rule, always from left to right, have been numbered, continue from the conclusion until a two-premiss rule with an unnumbered branch is reached.
4. From the root of the derivation, find a downmost leftmost rule instance that closes a hypothesis, and number its occurrences from the previously reached number on, and number the formulas down from these as in 2. Topmost unnumbered formulas are now conclusions of two-premiss rules.
5. Continue numbering formulas in the branches, left to right, until you come to a rule instance with a branch that has an unnumbered hypothesis. Repeat 3 and 4.

$$\begin{array}{c}
 \begin{array}{c}
 \begin{array}{c}
 \text{4. } \neg A \\
 \hline
 \text{5. } \perp \\
 \hline
 \text{6. } \neg\neg A
 \end{array}
 \end{array}
 \end{array}
 \begin{array}{c}
 \begin{array}{c}
 \text{1. } A \& B \\
 \hline
 \text{2. } A
 \end{array}
 \end{array}
 \begin{array}{c}
 \text{7. } \neg B \\
 \hline
 \text{8. } \perp \\
 \hline
 \text{9. } \neg\neg B
 \end{array}
 \begin{array}{c}
 \text{10. } \neg\neg A \& \neg\neg B \\
 \hline
 \text{11. } A \& B \supset \neg\neg A \& \neg\neg B
 \end{array}$$

1.	$A \& B$	hypothesis: goal $\neg\neg A \& \neg\neg B$
2.	A	1, $\&E$
3.	B	1, $\&E$
4.	$\neg A$	hypothesis: goal $\perp$
5.	$\perp$	4, 2, $\supset E$
6.	$\neg\neg A$	4-5, $\supset I$
7.	$\neg B$	hypothesis: goal $\perp$
8.	$\perp$	7, 3, $\supset E$
9.	$\neg\neg B$	7-8, $\supset I$
10.	$\neg\neg A \& \neg\neg B$	6, 9, $\&I$
11.	$A \& B \supset \neg\neg A \& \neg\neg B$	1-10, $\supset I$

Generalization: dependent type theory

Natural deduction rules are a special case:

$$\frac{A \quad B}{A \& B} \&I$$

$\&I : (A : \text{Prop}) \rightarrow (B : \text{Prop}) \rightarrow \text{Proof } A \rightarrow \text{Proof } B \rightarrow \text{Proof } (A \& B)$

In the general case, we need to decide what parts to show and how

- usually works by hiding a number of initial arguments - here, 2

Data structure for proof lines

10.	$\neg\neg A \& \neg\neg B$	6, 9, &I
11.	$A \& B \supset \neg\neg A \& \neg\neg B$	1–10, $\supset I$

(context, line_number, formula, premisses, rule_label, discharged)

Any of the parts may be suppressed, except either formula or rule_label

- should be possible to restore
- formalization of text inverts the suppression so that a tree can be constructed

Omitting proof lines: remove duplicates

(under a sufficiently strict criterion of equality)

- hypotheses shown only once
 - if corresponding to the same bound variable
 - if "close enough" to site of use
- conclusions shown only once
 - if corresponding to the same subterm of the proof term

Omitting proof lines: hide obvious steps

First ideas:

- tautologies, or single substitutions in Herbrand universe ("obvious" of M. Davis)
- what can be proven automatically (changes over time)

However, this can go too far:

- Sorites paradox ("adding one grain of sand does not yet make a heap")
- there are different audiences

Aggregation

Rules for iterated abstraction

$(\lambda l^1 : A_{Type}^1 \dots \lambda l^k : A_{Type}^k. M)_\tau$	▷	Let $l^1 : A^1$ ⋮ Let $l^k : A^k$ M We have proved τ
$(\lambda h^1 : A_{Prop}^1 \dots \lambda h^k : A_{Prop}^k. M)_\tau$	▷	Assume $A^1 (h^1)$ and ... and $A^k (h^k)$ M We have proved τ
$(\lambda x^1 : A_{Set}^1 \dots \lambda x^k : A_{Set}^k. M)_\tau$	▷	Choose arbitrarily x^1 in A^1, ..., x^k in A^k M Thus we have τ

Coscoy et al 1995

Aggregation: linguistic extensions in Informat

Let m be an integer.

Let n be an integer.

Suppose that m is prime.

Suppose that n is prime.

Aggregation: linguistic extensions in Informath

Let m be an integer.
Let n be an integer.
Suppose that m is prime.
Suppose that n is prime.



Let m and n be prime integers.

Other developments

Order of steps (underspecified by proof objects)

→ distance between premisses and conclusion

Discourse structure (Zinn, MALINCA)

- reminders, backward and forward references

Use clues from interaction and tactic-based proof sources

- to show what was given manually

Generate hypertext (for flexible information hiding)

Conclusion

Status

Done:

- grammar of statements, definitions, predicates, ...
- grammar of proof steps
- conversion from proof objects to sequences of texts
- some rules for ordering and omission
- small case studies (predicate logic + domain-specific rules)

To do:

- more ordering and omission
- hypertext
- larger case studies

This Is Not

The End